



Ordenação com Comparador Impreciso

Acúmulo de erro em quatro algoritmos de ordenação

Rafael Soares

Orientador: Prof. Tiago Almeida

Universidade Federal do Tocantins

Sumário

1. Contexto
2. Metodologia
3. Resultados
4. Conclusões

Duas formas de imprecisão na ordenação

Ajtai et al.: comparador impreciso

- A **comparação** falha quando $|a - b| \leq \delta$
- Nessa zona cega, a resposta é arbitrária
- Pergunta: quão longe da ordem correta o resultado pode ficar?
- Trabalho teórico, sem medições

Chen et al.: memória aproximada

- Os **dados** mudam sozinhos na memória (PCM aproximada)
- O comparador é exato; o valor lido é que está errado
- Métrica *Rem*: quantos elementos remover para o resto ficar ordenado
- Somente simulação, sem hardware real

Este trabalho usa o **modelo do Ajtai** e o **mede**, como o Chen faz com o dele.

Ajtai et al.: os erros se acumulam

- Exemplo do artigo: **bubble sort** sobre uma lista invertida em que vizinhos diferem exatamente δ
- Toda comparação cai na zona cega, então nenhuma troca acontece
- A saída é a lista invertida: o erro total é muito maior que δ

Pergunta central

Para um algoritmo real, o erro da saída fica limitado a alguns δ ou cresce com o tamanho da entrada?

Chen et al.: algoritmos reagem de forma diferente

Mesmo ponto de operação da memória aproximada:

Algoritmo	Desordem
Quicksort	$< 2\%$
Radix sort (LSD)	$\approx 1\%$
Mergesort	$\approx 55\%$

- **Mergesort:** cada fusão envolve mais elementos, e o erro se propaga até a última
- **Quicksort:** o particionamento confina o erro a subproblemas menores

O mesmo padrão aparece com comparador impreciso?

O comparador com tolerância δ

```
static inline int cmp_tol(int a, int b) {  
    long diff = (long) a - (long) b;  
    if (diff <= tol_delta && diff >= -tol_delta)  
        return 0; /* zona cega */  
    return (diff < 0) ? -1 : 1;  
}
```

- Todas as comparações entre elementos passam por `cmp_tol`
- Retorno 0: o algoritmo **não troca** os elementos
- δ lido da variável de ambiente `SORT_DELTA`

O algoritmo não conhece δ

Como no modelo do Ajtai, δ é propriedade do comparador. O algoritmo só pergunta e recebe -1 , 0 ou 1 .

Desenho do experimento

- **Algoritmos:** quicksort (dual pivot), mergesort (bottom-up), shellsort (Knuth), heapsort
- **Tolerâncias:** $\delta \in \{3, 5, 7, 11\}$, com $\delta = 0$ de referência
- **Tamanhos:** $n \in \{10^4, 10^5, 10^6\}$
- **Entradas:** uniformes em $[0, 10^6)$, 5 sementes
- **Versões:** serial e paralela (OpenMP, 4 threads)

Erro: 600 execuções. **Desempenho:** perf com $n = 10^6$, 30 repetições, núcleo fixo e ASLR desligado (1200 execuções).

Validação

Com $\delta = 0$, o vetor final é **idêntico byte a byte** ao da versão original nos 72 casos testados (4 algoritmos $\times$ 2 versões $\times$ 3 tamanhos $\times$ 3 sementes).

Métricas de erro da saída

rem elementos a remover para o resto ficar ordenado: n menos a maior subsequência não decrescente (métrica de Chen et al.)

inv número de inversões: pares $i < j$ com $a_i > a_j$

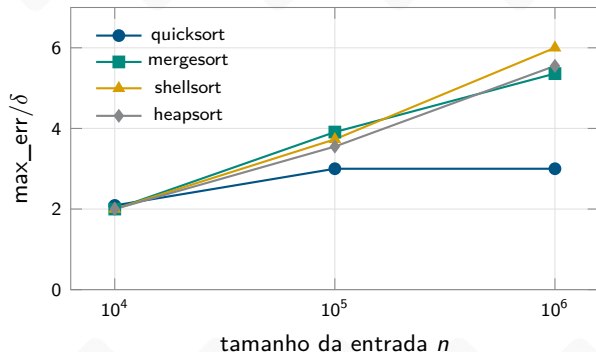
max_err o pior erro em valor: $\max_{i < j} (a_i - a_j)$

Por que max_err

É a grandeza que Ajtai et al. limitam: um bom algoritmo garante $\max_err \leq k \delta$ para um k pequeno e **fixo**. Se k cresce com n , o erro está se acumulando.

As métricas são das execuções seriais: com o mesmo δ e a mesma semente, serial e paralelo produziram exatamente os mesmos valores.

Resultado principal: o pior erro em múltiplos de δ



- **Quicksort** trava em 3δ a partir de $n = 10^5$
- Os outros três **crescem com n** : de 2δ até 5 a 6δ
- O acúmulo de erro descrito pelo Ajtai, observado na prática

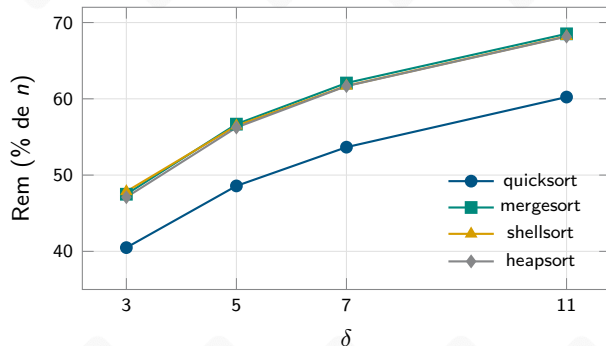
Pior caso entre 5 sementes e $\delta \in \{3, 5, 7, 11\}$.

O pior erro, em detalhe

Algoritmo	$\max_err/\delta$ (faixa entre os quatro δ)		
	$n = 10^4$	$n = 10^5$	$n = 10^6$
Quicksort	1,0 – 2,1	3,0	3,0
Mergesort	1,7 – 2,0	2,8 – 3,9	5,0 – 5,4
Shellsort	2,0	3,0 – 3,7	5,0 – 6,0
Heapsort	1,3 – 2,0	2,7 – 3,6	5,0 – 5,6

- Mesmo mecanismo que Chen et al. apontam para o mergesort: as fusões sucessivas propagam o erro; o particionamento do quicksort o confina
- Shellsort e heapsort também movem elementos por longas distâncias com base em comparações já imprecisas

Desordem pela métrica Rem



- $n = 10^6$, média de 5 sementes
- Quicksort de novo o menos desordenado: 40% contra 47% com $\delta = 3$
- Os outros três ficam praticamente sobrepostos

Cuidado: Rem depende da densidade dos dados

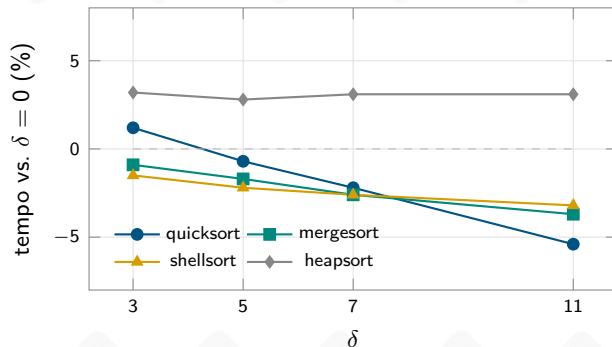
δ	Rem do quicksort		
	$n = 10^4$	$n = 10^5$	$n = 10^6$
3	1,5%	12%	40%
11	5,2%	30%	60%

- Valores em $[0, 10^6]$: com $n = 10^6$, o espaçamento médio entre vizinhos é 1
- Cada elemento tem $\approx 2\delta$ vizinhos dentro da zona cega
- O que importa é δ em relação ao espaçamento, não δ sozinho

Rem alto não significa saída ruim

Rem de 60% parece um desastre, mas o pior erro é de 33 numa faixa de 10^6 . O modelo do Ajtai gera muitos erros pequenos; Rem foi pensado para poucos erros grandes (Chen).

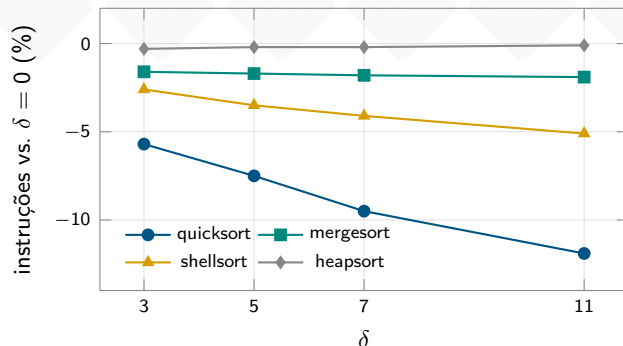
Desempenho: tempo de execução



- Serial, $n = 10^6$, mediana de 30 repetições com perf
- Quanto maior δ , **mais rápido**: quicksort $-5,4\%$, mergesort $-3,7\%$, shellsort $-3,2\%$ com $\delta = 11$
- **Heapsort** é a exceção: cerca de $+3\%$ para qualquer $\delta > 0$
- Paralelo (4 threads): mesmo padrão, até -7%

Variação entre repetições abaixo de 1% (heapsort: até 4%).

Por que fica mais rápido: menos trabalho



- Na zona cega o comparador responde “iguais” e o algoritmo **não troca**: menos movimentos de dados
- Quicksort: até -12% de instruções, porque também deixa de ordenar a partição do meio quando não distingue os pivôs
- Heapsort: instruções iguais, mas $+4\%$ de ciclos e até $+10\%$ de *cache misses*

Custo e benefício

O quicksort é ao mesmo tempo o mais preciso (3δ) e o que mais ganha em tempo.

Conclusões

- 1 Com comparador impreciso, **quicksort mantém o erro limitado** (3δ), enquanto mergesort, shellsort e heapsort **acumulam** erro com n
- 2 O padrão é o mesmo que Chen et al. observaram com memória aproximada, agora com o modelo de imprecisão do Ajtai
- 3 Rem é uma métrica enganosa para este modelo; o pior erro ($\max_err/\delta$) descreve melhor o comportamento
- 4 δ precisa ser interpretado em relação à densidade dos dados
- 5 A tolerância **reduz o tempo** (até 5,4% serial e 7,3% paralelo) em quicksort, mergesort e shellsort; o heapsort fica mais lento

Referências

- [1] M. Ajtai, V. Feldman, A. Hassidim, J. Nelson. *Sorting and Selection with Imprecise Comparisons*. ACM Transactions on Algorithms, 12(2), 2016.
- [2] S. Chen, S. Jiang, B. He, X. Tang. *A Study of Sorting Algorithms on Approximate Memory*. Proc. ACM SIGMOD, 2016.

Obrigado!

Rafael Soares

Universidade Federal do Tocantins